Autonomous Jeep Navigation in Roblox Using Proximal Policy Optimization

Reinforcement Learning — EC456

KV Modak - 23bcs067
Aneesh P - 23bcs095
Rishik Natra - 23bcs108
Janaki Ram R - 23bcs141
Indian Institute of Information Technology, Dharwad

April 2026

Abstract

This project demonstrates the application of deep reinforcement learning to train an autonomous vehicle agent within a 3D physics-based Roblox environment. A Proximal Policy Optimization (PPO) agent learns to navigate a jeep along a path, avoiding obstacles such as foliage and a river, to reach a designated target location. The system employs a client-server architecture where the Roblox game engine handles physics simulation and state observation, while a Python-based Flask server hosts the neural network brain that processes states and outputs discrete driving actions. After iterative reward shaping and architectural refinements, the agent achieved a peak success rate of **88%** over 920 episodes and 47,568 training steps.

Contents

1 Introduction **3**

2 Environment Setup **3**
2.1 System Architecture . . . 3
2.2 Roblox Environment . . . 3
2.3 Observation Space (State) . . . 4
2.4 Action Space . . . 4
2.5 Episode Termination Conditions . . . 4

3 Methodology **5**
3.1 Proximal Policy Optimization (PPO) . . . 5
3.2 Neural Network Architecture . . . 5
3.3 Reward Function Design . . . 5
3.3.1 Progress Reward . . . 5
3.3.2 Facing Reward . . . 6
3.3.3 Terminal Rewards . . . 6
3.4 Training Procedure . . . 6
3.5 Design Evolution . . . 7

4 Results **8**
4.1 Training Curves 8
4.2 Key Metrics 8
4.3 Analysis of Training Dynamics 8

5 Conclusion **9**
5.1 Future Work 9

1 Introduction

Autonomous navigation in 3D environments with physics-based dynamics presents significant challenges for reinforcement learning (RL) agents. Unlike grid-world or Atari benchmarks, continuous 3D environments involve complex vehicle dynamics, realistic collision physics, and spatially extended goal-seeking behavior.

This project explores training an RL agent to drive a jeep through a terrain containing obstacles (trees, rocks, and foliage), a river, and winding paths within Roblox Studio — a 3D game engine with real-time physics simulation. The agent must learn to:

1. Navigate toward a target location
2. Avoid collisions with environmental obstacles
3. Find an optimal path to goal
4. Maintain vehicle stability (avoid flipping)

We use **Proximal Policy Optimization (PPO)**, a state-of-the-art policy gradient algorithm known for its stability and sample efficiency, to train the agent in an online fashion via HTTP communication between the game environment and a Python training server.

2 Environment Setup

2.1 System Architecture

The system follows a client-server paradigm with two components communicating over HTTP: a client-server architecture. The Roblox game sends state observations via HTTP POST to the Python server every 0.15 seconds (∼400 requests/minute), which returns the selected action.

2.2 Roblox Environment

The environment consists of the following components:

- **Jeep (Agent):** A multi-part vehicle model with a `Chassis` as the root part. Movement is controlled via `BodyVelocity` and `BodyAngularVelocity` constraints applied to the chassis.
- **Map:** A terrain with a drivable road, off-road areas, and environmental obstacles under a `Map.Foilage` folder (trees, rocks, etc.).
- **Target Part:** The goal location. An episode is considered successful when the jeep's distance to the target falls below 15 studs.

2.3 Observation Space (State)

The agent receives an 11-dimensional state vector at each timestep:

Table 1: State vector components and their normalization.

Index	Feature	Raw Range	Normalization
0–5	Raycast sensor distances (6 directions)	$[0, 60]$ studs	$\div 60$
6	Distance to target	$[0, 1500]$ studs	$\div 1500$
7	Direction to target path	$[0, 1000]$ studs	$\div 1000$
8	Facing angle (dot product)	$[-1, 1]$	Raw
9	Current speed	$[0, 100]$ studs/s	$\div 100$
10	Curriculum level	$[1, 20]$	$\div 20$

The six raycast sensors are cast from the chassis at angles $\{-90, -45, -15, +15, +45, 0\}$ relative to the vehicle's forward direction, with a maximum range of 60 studs. These provide the agent with spatial awareness of nearby obstacles and path geometry.

2.4 Action Space

The agent selects from four discrete actions:

Table 2: Discrete action space.

Action	Name	Effect
0	Forward	Linear velocity = 75 along look vector
1	Left	Linear velocity = 35 along look vector, angular velocity = +4 rad/s
2	Right	Linear velocity = 35 along look vector, angular velocity = −4 rad/s
3	Stop	Zero velocity and angular velocity

2.5 Episode Termination Conditions

An episode terminates (with a reset to the start position) under any of the following conditions:

1. **Success:** Distance to target < 15 studs
2. **Off-path:** Guide distance > 10 studs for more than 10 consecutive seconds.
3. **Flipped:** Chassis `UpVector.Y` < 0.2 for more than 3 seconds
4. **Timeout:** Episode exceeds 60 seconds
5. **Collision:** Physical contact with foliage objects
6. **Front wall:** Forward raycast sensor < 3.0 studs
7. **Stuck:** Speed < 2.0 studs/s for more than 10 seconds

3 Methodology

3.1 Proximal Policy Optimization (PPO)

PPO is an actor-critic policy gradient method that constrains policy updates to a trust region using a clipped surrogate objective. Given a trajectory $\tau = (s_0, a_0, r_0, \ldots)$, PPO optimizes:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and $\hat{A}_t$ is the estimated advantage using Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{2}$$

The full loss function combines the policy loss, value loss, and an entropy bonus:

$$\mathcal{L} = \mathcal{L}^{\text{CLIP}} + c_1 \mathcal{L}^{\text{VF}} - c_2 H[\pi_\theta] \tag{3}$$

where $\mathcal{L}^{\text{VF}} = \frac{1}{2}(V_\theta(s_t) - V_t^{\text{target}})^2$ and $H[\pi_\theta]$ is the entropy of the policy distribution.

3.2 Neural Network Architecture

We employ a **separated actor-critic architecture** where the actor (policy) and critic (value function) have independent neural networks with no shared parameters. This design choice prevents gradient interference between the two objectives.

Key architectural decisions:

- **Tanh activations** instead of ReLU, provides bounded gradients and smoother optimization landscapes, which is beneficial for policy optimization.
- **Orthogonal initialization** with gain $\sqrt{2}$ for hidden layers and gain 0.01 for the actor output layer. The small output initialization ensures the initial policy is near-uniform across all actions, promoting balanced early exploration.
- **Separate optimizers** with independent learning rates for actor ($\alpha_{\text{actor}} = 10^{-3}$) and critic ($\alpha_{\text{critic}} = 10^{-3}$).

3.3 Reward Function Design

Reward shaping was critical to achieving fast convergence. We designed a dense, bounded reward function with four components, each constrained to a small numerical range to ensure the critic network can learn accurate value estimates:

$$r_t = r_t^{\text{progress}} + r_t^{\text{path}} + r_t^{\text{facing}} + r_t^{\text{delta}} \tag{4}$$

3.3.1 Progress Reward

Rewards movement toward the target:

$$r_t^{\text{progress}} = \text{clip}\left((d_{t-1} - d_t) \times 0.1,\ -1,\ 1\right) \tag{5}$$

where d_t is the Euclidean distance to the target at time t.

3.3.2 Facing Reward

Encourages the agent to face the target:

$$r_t^{\text{facing}} = \max(0,\ \vec{l_t} \cdot \hat{d}_t) \times 0.2 \tag{6}$$

where $\vec{l_t}$ is the vehicle's look vector and $\hat{d}_t$ is the unit direction toward the target.

3.3.3 Terminal Rewards

$$r_{\text{terminal}} = \begin{cases} +10.0 & \text{if } d_t < 15 \text{ (success)} \\ -2.0 & \text{if any failure condition is met} \end{cases} \tag{7}$$

The terminal rewards are deliberately moderate (not ± 50) to maintain a learnable value function. An excessively large terminal penalty causes episode returns to span several orders of magnitude, making the critic unable to fit accurate value estimates.

3.4 Training Procedure

Table 3: PPO hyperparameters.

Hyperparameter	**Value**
Actor learning rate α_{actor}	1×10^{-3}
Critic learning rate α_{critic}	1×10^{-3}
Discount factor γ	0.99
GAE parameter λ	0.95
PPO clip range ϵ	0.2
Entropy coefficient c_2	0.01
Value loss coefficient c_1	0.5
Rollout buffer size	512 steps
Mini-batch size	64
PPO epochs per update	10
Gradient clip (max norm)	0.5
KL early-stop threshold	0.1

The training procedure operates as follows:

Algorithm 1 Online PPO Training with Roblox Environment

```
Initialize actor π_θ and critic V_φ with orthogonal weights
Initialize rollout buffer B ← ∅
while training do
    Receive state s_t from Roblox via HTTP POST
    Compute reward r_t using Equation 4
    Sample action a_t ~ π_θ(·|s_t)
    Store (s_t, a_t, r_t, log π_θ(a_t|s_t), V_φ(s_t), done_t) in B
    Send a_t and reset signal back to Roblox
    if |B| ≥ 512 then
        Compute GAE advantages Â_t and returns R_t (Eq. 2)
        Normalize advantages: Â ← (Â − μ_A) / (σ_A + 10^−8)
        for epoch = 1 to 10 do
            Shuffle buffer indices
            for each mini-batch of size 64 do
                Compute clipped PPO loss (Eq. 1)
                Update θ, φ with gradient clipping at 0.5
            end for
            if mean KL divergence > 0.1 then
                break                                   ▷ Early stopping
            end if
        end for
        Clear buffer B
    end if
end while
```

3.5 Design Evolution

The final architecture was arrived at through iterative refinement. Key lessons learned include:

1. **Reward scale matters:** Early versions used terminal penalties of -50, creating episode returns in the range $[-12{,}000, 0]$. The critic loss remained at 6–18 (should be < 1), rendering advantage estimates meaningless. Bounding all rewards resolved this.

2. **Shared backbones cause interference:** The initial architecture used a shared feature extractor for both actor and critic. This caused the critic's gradient to corrupt actor weights and vice versa. Separating the networks resolved entropy stagnation at the uniform-random level ($H \approx \ln 4 \approx 1.386$).

3. **KL early-stopping must not be too aggressive:** A threshold of 0.02 stopped training on the first epoch of every update, preventing the policy from changing at all. Raising to 0.1 allowed meaningful policy improvement while still preventing catastrophic updates.

4. **Mini-batching dramatically improves sample efficiency:** With full-batch updates, only 30 gradient steps occurred over 11,000 training steps. Introducing 64-sample mini-batches with 10 epochs per update increased gradient steps by $\sim 80\times$.

4 Results

Training was conducted over **47,568 environment steps** across **920 episodes**. The following metrics were tracked:

4.1 Training Curves

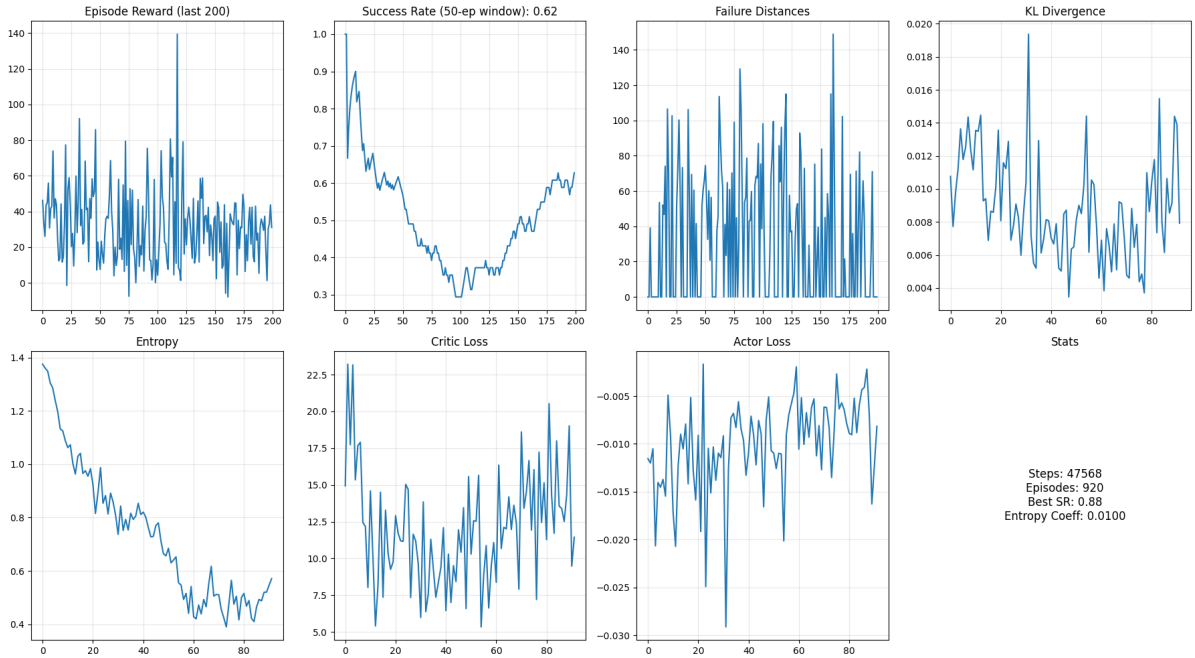

Figure 1: Training progress over 920 episodes. *Top row (L-R):* Episode reward, 50-episode rolling success rate, failure distances, KL divergence. *Bottom row (L-R):* Policy entropy, critic loss, actor loss, training statistics.

4.2 Key Metrics

Table 4: Summary of training results.

Metric	Value
Total training steps	47,568
Total episodes	920
Best success rate (50-episode window)	**88%**
Final success rate (50-episode window)	62%
Final policy entropy	~ 0.5 (from 1.386 initial)
KL divergence (mean)	~ 0.007
Training time	~ 2 hours

4.3 Analysis of Training Dynamics

- **Entropy Reduction (Fig. 1, bottom-left):** The policy entropy decreased from 1.386 (uniform random over 4 actions) to approximately 0.5 over 90 training updates, indicating that the agent learned to commit to specific actions rather than

acting randomly. The initial sharp drop corresponds to the agent learning to consistently drive forward.

- **Success Rate Trajectory (Fig. 1, top-center):** The 50-episode rolling success rate shows clear improvement from $\sim$ 30% to a peak of 88%, followed by some fluctuation. The fluctuation is characteristic of on-policy methods where exploring new strategies temporarily reduces performance.
- **Failure Distance Distribution (Fig. 1, top-right):** Failed episodes show a spread of failure distances. Early failures occur close to the start (distance > 100 studs from target), while later failures cluster at shorter distances, indicating the agent is progressing further along the path before failing.
- **KL Divergence Stability (Fig. 1, top-right):** The KL divergence remains below 0.02 for most training updates, confirming that policy updates are conservative and stable — a key benefit of the PPO clipping mechanism.

5 Conclusion

This project demonstrated that PPO can effectively learn autonomous navigation in a physics-based 3D environment with discrete action control. The key findings are:

1. **Reward design is the most critical factor:** Bounded, dense reward shaping with small terminal bonuses/penalties was essential for stable critic learning and, consequently, meaningful policy updates.
2. **Architectural separation matters:** Independent actor and critic networks with orthogonal initialization outperformed shared-backbone designs that suffered from gradient interference.
3. **Mini-batch PPO dramatically improves sample efficiency:** Even with the inherent latency of HTTP-based environment communication ($\sim$400 steps/minute), mini-batch SGD over the rollout buffer provided sufficient gradient updates for convergence.
4. The agent achieved a peak success rate of **88%**, learning to drive, steer, and avoid obstacles purely from scalar reward feedback without any human demonstrations or expert trajectories.

5.1 Future Work

- **Curriculum learning:** Progressively increasing path difficulty (longer paths, more obstacles, narrower roads) to build robust navigation skills.
- **Continuous action space:** Replacing discrete {Forward, Left, Right, Stop} with continuous throttle and steering angle for smoother control.
- **Visual observations:** Incorporating camera-based input (CNN feature extraction) instead of engineered raycast features.
- **Multi-agent training:** Running multiple jeep instances simultaneously to increase data throughput and reduce wall-clock training time.